REDROB · INTELLIGENT CANDIDATE DISCOVERY & RANKING CHALLENGE

Ranking 100,000 candidates the way a recruiter actually reads a resume

A title-gated hybrid ranker built CPU-only, no GPU, no network calls -- and the data-driven discovery that shaped its central design decision.

Harshdeep Singh

github.com/Harshdeep47

01 -- THE BRIEF

Rank 100K candidates the way a great recruiter would -- not by keyword match

Read the JD

Understand what the role actually needs, not just extract words from it.

Read the whole profile

Career history, skills, behavioral signals, platform activity -- the full picture.

Defend the shortlist

Deliver a ranked list a recruiter could actually trust and act on.

The trap the brief names explicitly: *"a candidate who has all the AI keywords listed as skills but whose title is 'Marketing Manager' is not a fit, no matter how perfect their skill list looks."*

02 -- EXPLORATION, NOT ASSUMPTION

Before writing a scorer, I read the data

100,000

candidate profiles, 1 fixed JD

75%

based in India; rest spread across 7 countries

133

distinct skill names -- a closed vocabulary

48

distinct job titles -- also closed

Two adversarial patterns surfaced immediately:

~5,500 candidates

non-technical titles (HR Manager, Sales Executive…) paired with templated "AI enthusiast" summary language -- the brief's keyword-stuffing trap, at scale.

68 honeypots

impossible profiles: "expert" skill with 0 months' use, or years-of-experience that doesn't add up against career history.

03 -- THE DISCOVERY THAT SHAPED EVERYTHING

Titles are trustworthy. Free text isn't -- not equally, anyway.

I checked whether each career_history entry's description actually matches its own title topically.

ML / AI titles

description content matches the role's own title

Non-technical titles

match rate -- the other 84% are shuffled in from an unrelated pool

Conclusion: title has to be the dominant, trustworthy signal for relevance. Free text (description, summary, self-reported skills) can confirm depth once relevance is established by title -- it cannot grant relevance on its own.

04 -- FOUND THE HARD WAY

v1 of the pipeline put an HR Manager in the top 100. Here's why.

CAND_0059448 -- rank #100, v1 output

"HR Manager at Wipro... directly relevant skills: Recommendation Systems (advanced), Sentence Transformers (advanced), FAISS (intermediate)"

Self-reported skills looked great. Career history (HR, then Content Writing) had nothing to do with ML/AI/search. v1 had no mechanism to weigh that.

THE FIX

1 **title_relevance_gate()**

Multiplicative gate over a closed taxonomy of all 48 titles: direct / adjacent / weak / none.

2 **Title-gated production_signal_score()**

Only trust career-history descriptions attached to a plausibly-relevant title.

3 **Title-gated reasoning quotes**

Never present a shuffled, untrustworthy description as evidence in the output.

05 -- ARCHITECTURE

Two stages: precompute once, rank in seconds

Why this split matters:

The spec time-boxes ranking to ≤5 min / ≤16GB / CPU-only / no network -- but not the work needed to get there. Splitting lets the expensive one-time cost (TF-IDF fit over 100K docs) sit outside the budget entirely, while the actual graded step stays a few seconds even at full scale.

Why TF-IDF, not a neural embedding model:

Tried sentence-transformers first. It pulls a multi-GB CUDA/PyTorch stack even for CPU-only inference -- exactly the dependency bloat that risks breaking a clean Docker rebuild. The vocabulary here is narrow and mostly closed (133 skills, 48 titles, one JD), so TF-IDF + cosine similarity does the job with zero exotic dependencies and fully auditable scores.

Additive for fit, multiplicative for disqualifiers

BASE SCORE (weighted sum, 0–1)

0.28	**Semantic similarity**	TF-IDF cosine vs. JD
0.22	**Core skill score**	verified assessment > self-report
0.16	**Production signal**	shipped-to-prod language, title-gated
0.12	**Availability**	recency, response rate, open-to-work
0.10	**Experience fit**	JD's 6–8yr ideal band
0.07	**Location fit**	Pune/Noida > other India > abroad
0.05	**Notice period fit**	sub-30-day preferred

MULTIPLICATIVE GATES (0–1, applied after)

title_relevance_gate — 0.05-1.0

the dominant gate -- see slide 4/5

visa_gate_penalty — 0.15

non-India + not willing to relocate

consulting_only_penalty — 0.35

ALL career history at consulting firms

cv_speech_only_penalty — 0.5

CV/speech-heavy, no NLP/IR depth

architecture_only_penalty — 0.5

senior, hasn't coded in 18+ months

title_chaser_penalty — 0.6

ladder-climbing via company-hopping

honeypot suppression — 0.01

impossible-profile sanity flags

07 -- VALIDATION

What the final output actually looks like

✓

100% Tier-S titles

every top-100 candidate currently holds a direct ML/AI/search role

✓

0 honeypots

in the top 100 (68 found across the full pool, all suppressed)

✓

0 keyword decoys

the ~5,500-candidate aspirational-AI trap, fully filtered

✓

97 / 100 India-based

3 non-India, all confirmed willing to relocate

✓

~4 sec runtime

full 100K pool, cached artifacts, <1GB RAM

✓

Deterministic

byte-identical output across repeated runs

What I'd change with more time

JD parsing is manual	Only one JD exists for this challenge, so I read it by hand. A real product needs an automated parser so the system generalizes without a code change.
TF-IDF won't generalize	Works well for this single, narrow-vocabulary JD. A real product would want a cached neural embedding step (still no GPU at rank time) with TF-IDF as a fallback.
Reasoning is templated, not LLM-polished	Deliberate, to guarantee zero hallucination under a no-network rank step. An offline, precompute-time LLM pass could improve phrasing without that risk.
Weights are reasoned, not learned	No ground truth was available to fit against. With labeled outcomes this becomes a learned ranker (logistic regression / GBM) over the same feature set.

Built, broken, and fixed in public -- see the git history.

Every architectural decision in this deck traces back to a specific exploration finding or a specific bug found in real output -- not a guess. The commit history shows the v1 bug, the root-cause investigation, and the fix, in that order.

github.com/Harshdeep47/redrob-candidate-ranking